# A Human-in-the-Loop Workflow for Reproducible LLM Coding of Legal Delegation and Administrative Discretion

Draft for discussion


## Abstract

The growing use of large language models (LLMs) for social-science coding creates a methodological tension: models can accelerate document analysis, but a single prompt obscures how concepts are operationalized, how intermediate judgments affect final labels, and where expert correction enters the process. We present Delegation Workflow Studio, a research prototype for coding congressional delegation and administrative discretion in U.S. financial-regulation laws. The prototype converts a progressively refined prompt family into an explicit, dependency-aware workflow whose stages, prompts, outputs, conditions, and validation checks are visible and editable. It supports two forms of human intervention: method-level revision of the codebook and workflow, and case-level inspection, correction, or re-evaluation of coded values with retained history. We document the development path from one-shot prompts to staged prompts and then to versioned workflow graphs, using a professor-provided historical coding as the operational ground truth for benchmark alignment. Preliminary binary results illustrate why class-specific evaluation matters: a looser prompt obtains higher raw accuracy on a highly imbalanced benchmark, whereas a stricter prompt substantially improves detection of the rare non-delegation class and balanced accuracy. The contribution is therefore not a claim that an LLM replaces expert coding. It is a design for making expert-guided computational coding inspectable, revisable, and reproducible.


## 1 Introduction

Researchers who study political institutions routinely convert complex legal and policy texts into structured variables (Grimmer and Stewart, 2013). In the present project, the substantive task is to identify whether a law delegates meaningful authority to an administrative actor and, conditional on delegation, to characterize the residual discretion available to that actor. Historically, such judgments have been made by trained researchers applying a coding protocol to legislative summaries. The approach is interpretable but costly to scale, especially when the goal is to compare many laws, preserve supporting evidence, and revisit classifications after the codebook changes.

LLMs offer a plausible acceleration mechanism: general annotation studies report substantial cost and performance advantages in some predefined tasks (Gilardi et al., 2023), and human–LLM frameworks have reduced effort in iterative qualitative coding (Dai et al., 2023). Prompt-only automation is nevertheless not a sufficient research design for a specialized legal construct. A long prompt may contain definitions, exceptions, stage ordering, and output rules, yet the resulting method remains difficult to audit. When a final rank is wrong, it may be unclear whether the model missed an administrative actor, over-counted a technical amendment, ignored a statutory constraint, or simply failed to follow the required ordering. Prompt changes can also improve one class while degrading another, an important concern when the benchmark distribution is highly imbalanced. Recent evidence on domain-expert legal coding likewise cautions that instruction-prompted generative models can perform poorly on complex constructs even when given the human codebook (Thalken et al., 2023).

This paper reports the design and preliminary evaluation of Delegation Workflow Studio, a human-in-the-loop research prototype that treats the *workflow* rather than an isolated prompt as the unit of the coding method. The workflow exposes ordered stages, dependencies, typed intermediate variables, deterministic branches, and final outputs. Researchers can edit prompts and node settings, test a workflow on a law, inspect a node-level trace, publish a versioned snapshot, and correct selected case-level outputs without silently rewriting the underlying model result.

The paper makes four contributions:

1. It reconstructs the methodological progression from one-shot classification to staged, dependency-aware coding of delegation and discretion.
2. It describes a visual workflow representation that makes the operational codebook editable by domain researchers and preserves intermediate evidence for audit.
3. It defines two explicit human loops: method-level workflow refinement and case-level correction or targeted re-evaluation.
4. It presents preliminary benchmark evidence showing that raw accuracy can misrepresent progress under severe class imbalance, motivating class-specific recall, balanced accuracy, and law-level disagreement review.

## 2 Research Setting

### 2.1 Delegation and discretion

The project concerns congressional delegation to administrative actors in financial-regulation legislation. The substantive foundation follows a transaction-cost account of delegation

(Epstein and O'Halloran, 1999) and prior quantitative analysis of U.S. financial-market regulation (Groll et al., 2021). The prompt family defines a delegation as a grant of authority to implement, administer, enforce, supervise, regulate, interpret, approve, deny, waive, certify, investigate, or issue rules. The later benchmark-aligned definition is deliberately narrower: a law counts as delegation only when it creates *new* authority or *materially expands* existing authority. Mere agency mentions, reporting duties imposed only on private actors, conforming amendments, and reductions of existing regulation are excluded.

Delegation and discretion are related but distinct concepts. Delegation asks whether an administrative actor receives meaningful authority. Discretion asks how much residual choice remains after statutory goals, procedures, formulas, oversight, deadlines, reporting requirements, review provisions, and other constraints are taken into account. The current ordinal coding uses five ranks:

**0 — No delegation.**
No qualifying authority is delegated.
**1 — Narrow or ministerial.**
Duties are substantially specified and leave little policy choice.
**2 — Bounded discretion.**
Choice exists but is structured by criteria, formulas, procedures, or oversight.
**3 — Meaningful policy choice.**
The actor can make consequential choices within a recognizable statutory framework.
**4 — Broad policy-shaping authority.**
Authority is durable and wide, with relatively few meaningful constraints.

This formulation follows the language embedded in the professor-provided prompts, capturing the structural distinctions between ministerial duties (Rank 1), bounded decision-making (Rank 2), policy choices (Rank 3), and broad delegation (Rank 4).

## 2.2 Benchmark and source discipline

The operational benchmark is a professor-provided historical coding of `DelegateLaw` and `Discretion_Rank`. We refer to it as the project's *ground truth* for benchmark alignment, while recognizing that it represents a historical social-science coding rather than an adjudication of legal truth.

The source universe is methodologically important. The historical labels were produced from CQ summaries or corresponding descriptions of major provisions. Full statutory text can contain technical or minor delegations absent from those summaries. Consequently, a full-text model may be legally plausible while disagreeing with the historical label for reasons unrelated to prompt quality. Benchmark runs therefore use the same summary-level information set; full-text runs are labeled exploratory.

The repository currently contains 169 summary files (161 delegation-positive and eight delegation-negative) spanning 1950–2013, which represent the intersection of historical legislative records and available summaries. The broader, unlabeled document database in the prototype extends through 2020 (totaling 412 documents), while the ground-truth benchmark CSV contains 121 manually coded financial laws spanning 1950–2013.

# 3 Related Work

## 3.1 LLMs as research coders

LLMs have shown promise as low-cost annotators. Gilardi et al. (2023) report that ChatGPT outperformed crowd workers on several text-annotation tasks, while Dai et al. (2023) demonstrate an iterative human–LLM process for thematic analysis. Human-centric work has also emphasized assisting coders rather than treating full automation as the only objective, particularly when rare codes and coder-specific practices matter (Spinoso-Di Piano et al., 2023).

These results do not establish reliability for specialized legal constructs. Legal NLP benchmarks such as LexGLUE and LegalBench emphasize task-specific evaluation and expert-designed categories (Chalkidis et al., 2022; Guha et al., 2023). More directly, Thalken et al. (2023) find that prompted generative models performed poorly when applying a domain-expert codebook to historical Supreme Court opinions, with their strongest results instead coming from an in-domain fine-tuned model. This contrast motivates our conservative position: LLM coding is evaluated against a controlled benchmark and remains subject to expert review.

## 3.2 Expert supervision and workflow authoring

Weak supervision provides one model for converting expert heuristics into reusable computational signals (Ratner et al., 2017); technology-assisted review in legal discovery likewise demonstrates iterative human feedback over ranked documents (Cormack and Grossman, 2014). Our current prototype does not learn a weak-supervision label model and does not implement a formal active-learning queue. It instead operationalizes the expert codebook directly as a versioned executable workflow.

Visual prompt-chaining systems are important predecessors. AI Chains decomposes LLM tasks into modular, controllable steps (Wu et al., 2022b), while PromptChainer provides node-edge authoring, per-step testing, intermediate-output inspection, and debugging for chained prompts (Wu et al., 2022a). We therefore do not claim to invent human-in-the-loop annotation or visual prompt chaining. Our contribution is their integration into a research instrument for law-level delegation and ordinal discretion coding, with typed institutional variables, deterministic branches, benchmark mapping, source-policy controls, document-level traces, and separate method- and case-level interventions.

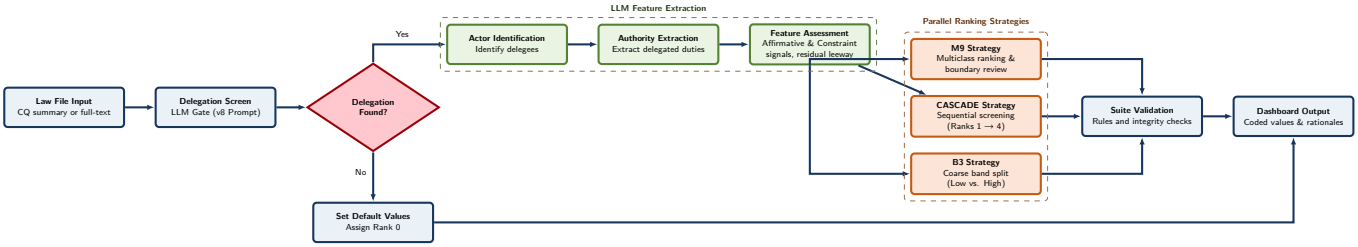


**Figure 1:** Modular architecture of the Delegation Workflow Studio workflow graph (`Version 4 - Detailed`). Upstream outputs flow into parallel CASCADE (sequential), M9 (multiclass), and B3 (coarse split) ranking strategies, followed by validation checks and final output serialization.

# 4 From Prompts to an Explicit Workflow

The system evolved through repeated attempts to make the coding rule both more accurate and more interpretable. Table 1 groups the many prompt files into four methodological phases.

## 4.1 Why a workflow rather than a larger prompt?

The final rank depends on earlier judgments. A model should first determine whether a qualifying delegation exists, then inventory administrative actors and authority, assess the breadth and centrality of that authority, identify constraints, and only then estimate residual leeway. If no delegation exists, the workflow assigns rank 0 deterministically instead of spending another model call on an inapplicable question. If delegation exists, the rank stage receives the earlier values and rationales explicitly.

This dependency structure addresses a recurrent failure of independent-column coding: a later variable such as `discretion_rank` cannot reliably use an earlier delegation decision unless the pipeline stores and forwards it. In the prototype, execution order is computed from the graph rather than assumed from the visual order of columns. Each node records inputs, outputs, status, timing, and, where applicable, evidence and rationale.

## 4.2 Workflow primitives

The implemented engine supports document-input, LLM, condition, deterministic set-value, validation, output, and rank-descriptor nodes. A workflow definition stores its nodes, edges, typed outputs, and metadata as a database-managed artifact (represented visually in Figure 1). Researchers can create a workflow from a template, duplicate it, import or export its JSON representation, edit its draft, validate it, test it, and publish a numbered snapshot with a changelog and definition hash.

The distinction between *workflow* and *campaign* is central. A workflow defines the research instrument: concepts, prompts, stages, dependencies, branching, and outputs. A campaign or evaluation run applies a selected definition to a document collection under a specified model policy and stores the resulting dataset and traces.

# 5 Human-in-the-Loop Design

Figure 2 presents the proposed research process. Solid arrows correspond to implemented or directly supported interactions. The dashed arrow denotes a planned governance step in which selected case-level lessons are deliberately promoted into a new workflow version; corrections do not currently retrain or silently alter the method.

## 5.1 Method-level intervention

The Stage Builder exposes source-text policy, evaluation mode, calibration settings, each stage's purpose and prompt, typed output fields, and whether a field is final or retained only for audit. A live preview shows the compiled prompt. The more general graph editor exposes upstream-field selection, conditional routing, deterministic assignments, validations, and rank-specific criteria. This lets a domain expert revise the operational codebook without editing application source code.

Publishing creates an immutable numbered definition snapshot, but the current prototype does not implement a separate professor-approval request or sign-off state. Any authorized editor can publish. Similarly, the current user interface does not yet provide a complete published-version comparison and pinning workflow for all campaign types. We therefore describe publishing as a reproducibility primitive, not as a completed governance protocol.

## 5.2 Case-level intervention

For an individual coded law, a researcher can inspect the final values and the node-by-node execution trace, manually override a selected value and rationale, or request AI re-evaluation with corrective feedback. History distinguishes initial AI output, manual override, and AI re-evaluation. A manual correction is local to that law; it does not automatically change the codebook, the prompt, or other cases.

This separation is scientifically useful. Case correction protects the dataset from a known error, whereas method revision changes the coding rule and should trigger a new version and controlled rerun. The prototype therefore avoids an opaque

**Table 1:** Development from prompt-only classification to a workflow research instrument.

| Phase | Representation | Methodological change | Observed motivation | Auditability |
|---|---|---|---|---|
| I | One-shot prompt | Joint binary delegation and 0–4 discretion prediction with rationale and evidence; enforce no-delegation ⇒ rank 0. | Establish a first end-to-end coding baseline. | Final rationale, but stage errors remain entangled. |
| II | Decomposed prompt | Separate label-only and rationale-bearing tasks; then separate delegation, amount/centrality, constraints, constraint categories, and rank. | Isolate output-design effects and identify where reasoning fails. | Stage text is visible, but ordering is still embedded in prompt files. |
| III | Coding-sheet-aligned staged prompt | Map stages to explicit variables; tighten definitions for agency creation, appointment, external audit, sunset provisions, financial-domain scope, and omnibus laws. | Reduce false positives and align outputs with the professor's historical coding sheet. | Intermediate variables are explicit but not yet a reusable executable graph. |
| IV | Versioned workflow | Convert prompts into typed nodes, upstream dependencies, conditions, deterministic assignments, validations, outputs, reusable templates, execution traces, and published snapshots. | Make the method human-editable, testable, and reproducible across documents and models. | Node-level values, rationales, timing, provenance, and branch decisions are inspectable. |

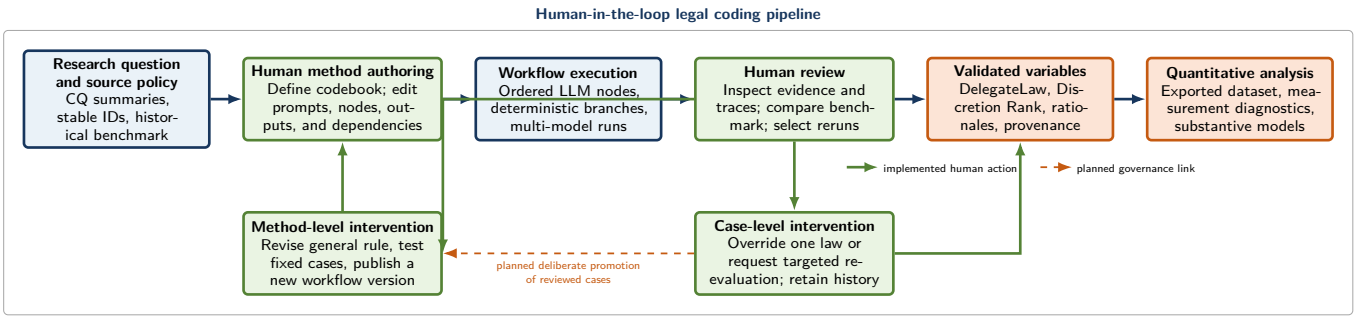


**Figure 2:** Human-in-the-loop legal coding pipeline. The prototype supports method-level intervention through editable workflows and case-level intervention through inspection, override, and targeted re-evaluation. A case correction changes that law's stored value; a separate, deliberate workflow revision is required to change the research instrument. Dashed elements denote planned extensions.

"learning" process in which corrections silently alter subsequent classifications.

### 5.3 Implemented boundary

The current implementation has two authoring surfaces. Builder-backed workflows present a readable sequence of domain stages whose prompts and output metadata can be edited, but their compiled graph is currently read-only and the stage sequence is determined by the selected mode. The generic graph editor permits lower-level node and edge authoring. Validation nodes currently report diagnostic flags but do not halt execution. Role-specific professor/research-assistant/student permissions, formal approval gates, automatic promotion of case corrections, and a hard-case review queue are future extensions.

## 6 Experimental Design

### 6.1 Controlled prompt comparison

The professor-authored testing protocol fixes a development set, holds source documents constant, records prompt and model versions, preserves law identifiers, and compares each output with the manual benchmark. After each revision, the same laws are rerun and disagreements are reviewed law by law. The protocol explicitly distinguishes recurring errors from isolated mistakes.

The recorded error taxonomy includes: treating any delegation as high discretion; treating minor technical delegation as central; missing constraints; treating rulemaking authority as unconstrained when standards or procedures exist; inventing authority not found in the supplied text; relying on full-text evidence absent from the summary; producing a final rank inconsistent with earlier stages; and confusing delegation with

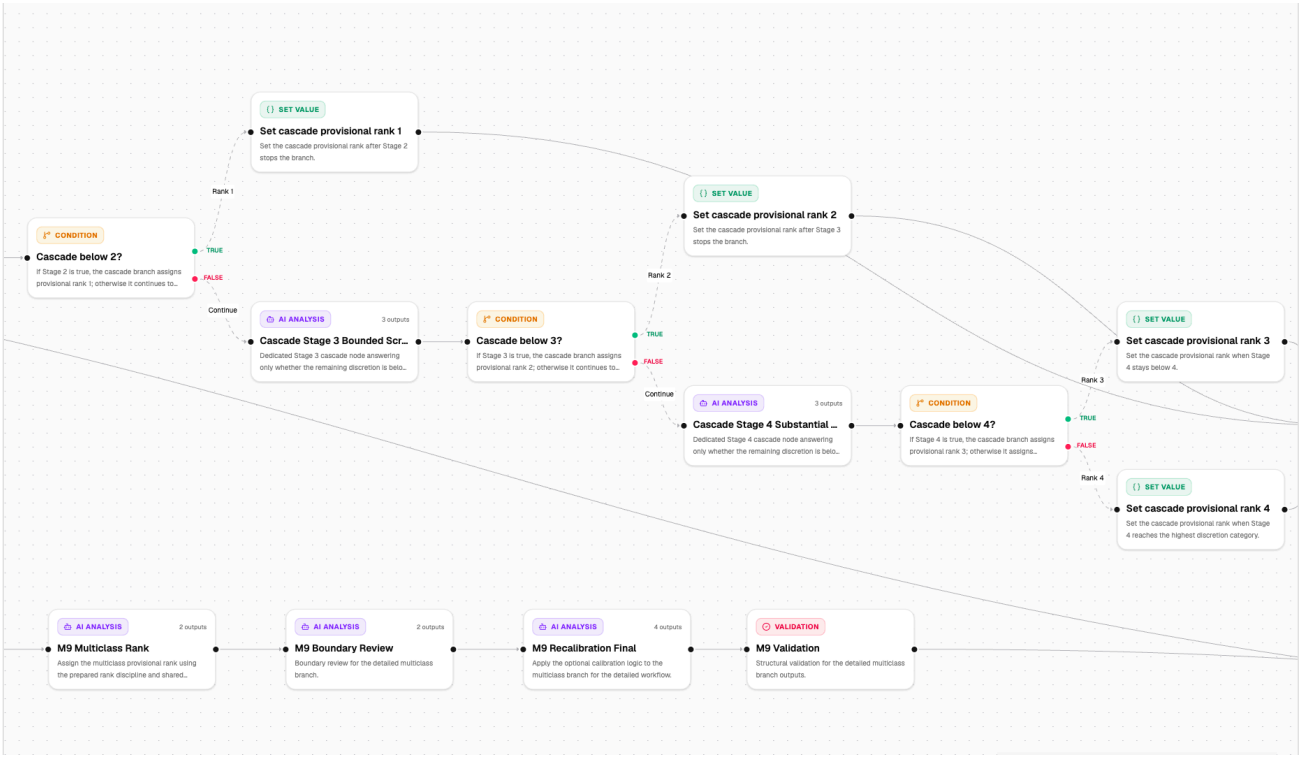


**Figure 3:** Product view of the workflow graph, showing how the discretion-rank task is decomposed into explicit LLM analysis nodes, condition nodes, deterministic set-value nodes, validation nodes, and connected branches. This interface exposes the coding method as an editable research instrument rather than a single hidden prompt.

discretion.

The canonical 15-law development set is composed of the 13 delegation-positive summaries available in the summary folder, supplemented by two delegation-negative cases from the broader corpus (PL96-3 and PL92-165) to support class-balanced evaluation.

## 6.2 Model and workflow comparison

The evaluation dashboard can lock one workflow definition, apply it to the same files across multiple selected models, retain model-keyed values and traces, and report per-variable matches, rank error, token use, cost, timing, and failures. This design separates the coding strategy from the model used to execute it.

The campaign evaluation interface can summarize model-level costs, token use, per-strategy benchmark agreement, mapped cells, and variable-level matches. Figure 4 shows a compact product view of this comparison for two selected models.

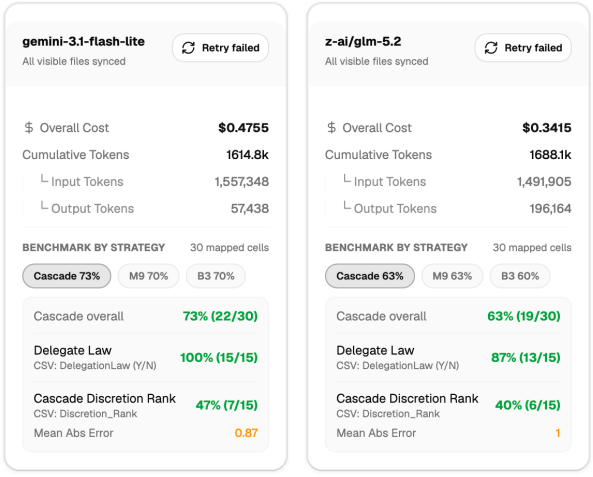


**Figure 4:** Benchmark-comparison interface showing model-level cost, token counts, strategy-level agreement, and variable-level matches for selected models.

## 6.3 Metrics

For the binary task, we report overall accuracy, positive-class recall, negative-class recall, and balanced accuracy. If the historical positive class is delegation, then

$$\text{BalancedAccuracy} = \frac{1}{2}\left(\frac{\text{TP}}{\text{TP}+\text{FN}} + \frac{\text{TN}}{\text{TN}+\text{FP}}\right).$$

For the ordinal discretion task, the evaluation interface supports exact accuracy, accuracy within one rank, mean absolute error (MAE), and a 0–4 confusion matrix. These measures distinguish an adjacent-rank disagreement from a substantively larger error.

# 7 Preliminary Findings

## 7.1 Binary delegation

Table 2 reconstructs metrics from persisted local dashboard rows. The benchmark is severely imbalanced: 161 positive cases and only eight negative cases. An always-positive classifier would obtain 95.27% raw accuracy while failing every negative case. Raw accuracy is therefore not an adequate measure of whether prompt refinement improves the underlying coding rule.

The looser v7 strategy yields the better raw accuracy because it preserves positive-class recall. The conservative v8 strategy makes more false-negative errors, but it detects seven of the eight rare negative cases and produces a much higher balanced accuracy. The substantive result is therefore a tradeoff, not a monotonic claim that one prompt is "more accurate." If the research priority is recovering rare non-delegation cases for expert review, v8 is the stronger baseline. If the priority is maximizing delegation recall, v7 is stronger.

## 7.2 A calibration failure

The law PL83-577 illustrates why source discipline and intermediate evidence matter. The historical CSV codes it as non-delegation with rank 0. An exploratory staged full-text run instead identified Securities and Exchange Commission rulemaking and exemption authority and returned delegation with rank 3. This disagreement could reflect an overbroad coding rule, a source-universe difference, or a mismatch between historical summary coding and full statutory detail. A final label alone cannot distinguish these explanations; a trace that records actor, authority, source evidence, and stage decisions can.

## 7.3 Discretion rank and model comparison

Early 15-law experiments provide exploratory ordinal results (Table 3). Exact agreement remained modest, reaching 40% in the strongest checked runs, while 73.3% of predictions were within one rank across all four strategies. These small-sample results motivated clearer separation of delegation screening, constraint extraction, and final ranking. They should not be treated as a held-out estimate of final workflow performance.

The production interface also supports multi-model comparisons. The cross-model evaluation results on the 15-law development set are summarized in Table 4. Across the six models, CASCADE generally provides the strongest exact-accuracy results, while M9 often improves within-one-rank accuracy and mean absolute error. B3 generally performs worse on this development set, but the pattern should be interpreted cautiously because the evaluation sample is small. The strongest exact-accuracy result is gemini-3.5-flash with CASCADE (50.0%), while the lowest MAE in Table 4 is gemini-3.1-flash-lite with M9 (0.800).

# 8 Discussion

## 8.1 The workflow as a research instrument

The main contribution of the prototype is methodological rather than a claim of universally superior classification. The workflow externalizes decisions that were previously buried inside prompt prose: which source text is admissible, which stages must precede others, which intermediate variables are retained, when deterministic rules apply, and what information the final rank is allowed to use. The resulting artifact can be inspected, duplicated, versioned, tested, and rerun.

This design is aligned with a quantitative social-science perspective. The goal is not merely to produce plausible legal interpretations. It is to operationalize a concept consistently, compare predictions with a coded benchmark, diagnose systematic disagreements, and preserve enough provenance for another researcher to reproduce the coding process. In that sense, the workflow plays a role analogous to a versioned coding protocol or measurement instrument.

## 8.2 What the human contributes

The human remains responsible for defining constructs, choosing the source universe, approving the codebook in the substantive sense, interpreting benchmark disagreements, correcting known errors, and deciding whether a case-level lesson warrants a method-level revision. The LLM contributes scalable extraction, structured intermediate judgments, and candidate classifications. The system contributes orchestration, provenance, comparison, and versioning.

The two human loops should not be conflated. A local override is appropriate when a researcher knows that one case is wrong. A workflow revision is appropriate when the error reveals a general rule. The latter should create a new version and be evaluated on a controlled set before adoption.

## 8.3 Limitations

Several limitations bound the current evidence. First, the binary benchmark is extremely imbalanced, and prompt development on the same cases risks overfitting. Second, the benchmark is based on summaries; conclusions do not automatically transfer to full statutes. Third, no verified intercoder-reliability information is currently available for the historical labels. Fourth, a single formal run per strategy does not establish statistical stability, even if spot reruns return identical outputs. Fifth, ordinal ranks can propagate errors from earlier stages. Sixth, the current study concerns one policy domain and has no external-dataset validation. Seventh, model and provider versions can drift. Finally, some audit and governance capabilities remain incomplete: validation is diagnostic, actor-attributed override logs are limited, and workflow-version approval and pinning are not yet uniform across the product.

**Table 2:** Preliminary binary delegation results reconstructed from persisted dashboard rows. Positive denotes the historical delegation-positive class.

| Strategy | N | TP | FN | TN | FP | Accuracy | Pos. recall | Neg. recall | Balanced acc. |
|---|---|---|---|---|---|---|---|---|---|
| Prompt v7, Stage 1 v4 | 169 | 156 | 5 | 4 | 4 | 94.67% | 96.89% | 50.00% | 73.45% |
| Prompt v8 | 169 | 148 | 13 | 7 | 1 | 91.72% | 91.93% | 87.50% | 89.71% |

Model: `gemini-3.1-flash-lite-preview`. Runs were executed on June 20, 2026. The v8 subsets (8 negative and 161 positive documents) are combined here as a single evaluation series.

**Table 3:** Exploratory Discretion Rank results on 15 laws. All runs used `gemini-3.1-flash-lite-preview`.

| Dashboard strategy | Exact | Within 1 | MAE |
|---|---|---|---|
| Prompt v3 | 26.7% | 73.3% | 1.067 |
| Prompt v3.4 | 40.0% | 73.3% | 0.933 |
| Prompt v3.5 | 40.0% | 73.3% | 0.933 |
| Prompt v3.6 | 33.3% | 73.3% | 1.000 |

Metrics were reconstructed from persisted dashboard rows matched to the ground-truth CSV by public-law number. Campaigns v3.4, v3.5, and v3.6 share the identical prompt text (Prompt Version 7.2); variance in metrics is due to LLM nondeterminism.

### 8.4 Novelty claim

Related systems separately study LLM annotation, active review, weak supervision, prompt chaining, and visual workflow authoring (Cormack and Grossman, 2014; Ratner et al., 2017; Wu et al., 2022b,a). Our cautious claim is that Delegation Workflow Studio offers a potentially distinctive integration for expert-guided legal-institutional coding: a domain codebook represented as an editable dependency graph, deterministic and generative stages in one instrument, trace-linked benchmark comparison, and both method-level and case-level human intervention. A broader product comparison is still required before claiming industry uniqueness.

## 9 Replication Workflow

The current prototype can be replicated through the following research-facing sequence:

1. **Prepare the corpus.** Upload CQ summaries or corresponding major-provisions text and retain stable law identifiers. Keep full statutes in a separately labeled exploratory collection.
2. **Create the instrument.** Duplicate the delegation-and-discretion workflow template. Review every stage's purpose, prompt, required inputs, typed outputs, and final/internal visibility.
3. **Validate and test.** Run representative positive, negative, and boundary cases. Inspect graph validation, final outputs, and node-level traces.
4. **Freeze a version.** Save a changelog and publish a numbered workflow snapshot before a benchmark run. Linking a campaign copies the workflow's draft definition and revision directly to the dashboard, preventing subsequent edits from silently affecting active runs.
5. **Run a controlled evaluation.** Apply the same workflow and source set across selected models or strategies. Record model identifiers, settings, timestamps, failures, token use, and cost.
6. **Compare with ground truth.** Map benchmark columns, calculate binary class-specific metrics and ordinal rank metrics, and inspect every mismatch.
7. **Intervene explicitly.** Correct a case locally when appropriate. If the disagreement reveals a general rule, revise the workflow separately, publish a new version, and rerun the fixed development set.
8. **Export the dataset.** Preserve final variables alongside the workflow version, model, source universe, and audit history needed for downstream quantitative analysis.

## 10 Future Work

Immediate empirical work should reconcile the benchmark artifacts, export the live multi-model results, evaluate discretion-rank performance, and repeat runs to characterize model variability. Methodological work should add a held-out evaluation set and, if feasible, independent recoding to estimate intercoder reliability.

Product work should add role-specific governance, workflow-version comparison and pinning, model-aware manual correction in the evaluation dashboard, editable validation rules, and a deliberate mechanism for promoting reviewed cases into a new calibration set or workflow version. External validity should be tested on an additional policy dataset before making claims beyond financial regulation.

## 11 Conclusion

This project began as an effort to improve prompts for classifying legal delegation and discretion. Its more consequential development was the recognition that prompt wording alone cannot provide the transparency, dependency management, and reproducibility required for a research coding method. The resulting prototype turns the codebook into an explicit workflow and places human judgment at two visible points: designing the method and correcting cases. Preliminary evidence shows why this structure is necessary: prompt revisions change class-specific trade-offs that raw accuracy can conceal. The next step is not to remove the human from the loop, but to make expert intervention systematic, auditable, and empirically evaluable.

**Table 4:** Cross-model evaluation results on the 15-law development set for CASCADE, M9, and B3 ranking strategies.

| Model | Strategy | Exact Accuracy | Within 1 Rank | MAE | N |
|---|---|---|---|---|---|
| gemini-3.5-flash | CASCADE | 50.0% | 57.1% | 0.929 | 14 |
| | M9 | 21.4% | 71.4% | 1.071 | 14 |
| | B3 | 28.6% | 64.3% | 1.143 | 14 |
| gemini-3.1-flash-lite | CASCADE | 46.7% | 73.3% | 0.867 | 15 |
| | M9 | 40.0% | 80.0% | 0.800 | 15 |
| | B3 | 40.0% | 60.0% | 1.067 | 15 |
| qwen/qwen3-235b-a22b-2507 | CASCADE | 46.7% | 60.0% | 1.133 | 15 |
| | M9 | 26.7% | 73.3% | 1.133 | 15 |
| | B3 | 26.7% | 46.7% | 1.533 | 15 |
| z-ai/glm-5.2 | CASCADE | 40.0% | 73.3% | 1.000 | 15 |
| | M9 | 40.0% | 73.3% | 1.000 | 15 |
| | B3 | 33.3% | 60.0% | 1.133 | 15 |
| deepseek-v4-flash | CASCADE | 40.0% | 73.3% | 0.933 | 15 |
| | M9 | 26.7% | 80.0% | 1.067 | 15 |
| | B3 | 26.7% | 40.0% | 1.667 | 15 |
| nousresearch/hermes-3-llama-3.1-405b | CASCADE | 33.3% | 53.3% | 1.333 | 15 |
| | M9 | 33.3% | 60.0% | 1.333 | 15 |
| | B3 | 26.7% | 60.0% | 1.333 | 15 |

# References


Ilias Chalkidis, Abhik Jana, Dirk Hartung, Michael Bommarito, Ion Androutsopoulos, Daniel Katz, and Nikolaos Aletras. LexGLUE: A benchmark dataset for legal language understanding in english. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, pages 4310–4330. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.acl-long.297.

Gordon V. Cormack and Maura R. Grossman. Evaluation of machine-learning protocols for technology-assisted review in electronic discovery. In *Proceedings of the 37th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 153–162. ACM, 2014. doi: 10.1145/2600428.2609601.

Shih-Chieh Dai, Aiping Xiong, and Lun-Wei Ku. LLM-in-the-loop: Leveraging large language model for thematic analysis. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 9993–10001. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.findings-emnlp.669.

David Epstein and Sharyn O'Halloran. *Delegating Powers: A Transaction Cost Politics Approach to Policy Making under Separate Powers*. Cambridge University Press, Cambridge, 1999.

Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. Chatgpt outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30): e2305016120, 2023. doi: 10.1073/pnas.2305016120.

Justin Grimmer and Brandon M. Stewart. Text as data: The promise and pitfalls of automatic content analysis methods for political texts. *Political Analysis*, 21(3):267–297, 2013. doi: 10.1093/pan/mps028.

Thomas Groll, Sharyn O'Halloran, and Geraldine McAllister. Delegation and the regulation of u.s. financial markets. *European Journal of Political Economy*, 70:102058, 2021. doi: 10.1016/j.ejpoleco.2021.102058.

Neel Guha, Julian Nyarko, Daniel E. Ho, Christopher Ré, Adam Chilton, et al. LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023. arXiv:2308.11462.

Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. Snorkel: Rapid training data creation with weak supervision. *Proceedings of the VLDB Endowment*, 11(3):269–282, 2017. doi: 10.14778/3157794.3157797.

Cesare Spinoso-Di Piano, Samira Rahimi, and Jackie Cheung. Qualitative code suggestion: A human-centric approach to qualitative coding. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 14887–14909. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.findings-emnlp.993.

Rosamond Thalken, Edward Stiglitz, David Mimno, and Matthew Wilkens. Modeling legal reasoning: LM annotation at the edge of human agreement. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9252–9265. Association for

Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.575.

Tongshuang Wu, Ellen Jiang, Aaron Donsbach, Jeff Gray, Alejandra Molina, Michael Terry, and Carrie J. Cai. Promptchainer: Chaining large language model prompts through visual programming. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts*. ACM, 2022a. doi: 10.1145/3491101.3519729.

Tongshuang Wu, Michael Terry, and Carrie Jun Cai. Ai chains: Transparent and controllable human-ai interaction by chaining large language model prompts. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. ACM, 2022b. doi: 10.1145/3491102.3517582.